Source Code

< Back to *The Program* | Forward to *Program Output* >

```
/* start read.c */

/*****************************************************************************
*  Name.......: read.c                                                       *
*                                                                            *
*  Description: C program to test text file reading.                         *
*                                                                            *
*  Author.....: Scott Brueckner (99001; COP2222 E002; Tue, 7:00-9:45 pm)     *
*                                                                            *
*  Date.......: 11/09/1999                                                   *
*                                                                            *
*  Arguments..: int   argc   = Count of command line arguments               *
*               char *argv[] = Array of pointers to command line arguments   *
*                              argv[0] = path to executable                  *
*                              argv[1] = name of file to process             *
*                              argv[2] = file open mode                      *
*                                        t = text                            *
*                                        b = binary                          *
*                              argv[3] = read function                       *
*                                        fscanf, fgets, fgetc, fread         *
*                                                                            *
*  Return.....: int: 0 = normal completion                                   *
*                    1 = error occurred                                      *
*                                                                            *
*  Compilers..: Visual C++ 6.0   (Win32)                                     *
*               Borland C++ 4.52 (16-bit DOS)                                *
*               GNU gcc 2.7.2.1  (Linux)                                     *
*                                                                            *
*  Notes......: This program opens the text file specified in argv[1] in the *
*               mode (text or binary) specified in argv[2] and reads it using*
*               the C function specified in argv[3]. It prints out the       *
*               results line-by-line in a modified "hex dump."               *
*                                                                            *
*               The allowable C read functions (fscanf(), fgets(), fgetc(),  *
*               and fread()) are implemented in separate functions in this   *
*               program (T_fscanf(), T_fgets(), T_fgetc(), and T_fread(),    *
*               respectively).                                               *
*                                                                            *
*               All of the functions (except T_fread()) are called by        *
*               dereferencing a pointer to the appropriate function. Each    *
*               function (except T_fread()) returns the next line of the text*
*               file, and the overall logic is controlled in main(). The     *
*               T_fread() function is "stand-alone" and contains its own     *
*               logic for traversing the file.                               *
*                                                                            *
*               This is an academic exercise for demonstration and testing   *
*               purposes. It contains some limitations that are inappropriate*
*               in the "real world." These are noted in the comments. Also,  *
*               this program doesn't "do" anything with the information in   *
*               the text file; it simply displays exactly what it read.      *
*                                                                            *
*               The sample text files included with this program are examples*
*               of delimited data files. In a "real" application, you would  *
*               need to perform additional processing, such as removing      *
*               carriage-return and line-feed characters, splitting the lines*
*               into individual fields, removing the quotes from the text    *
*               strings, and validating the resulting data. After that, you'd*
*               likely need to write the data back to disk in some other     *
```

```
*               format, such as a specific database.                          *
********************************************************************************/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

#define CR 13            /* Decimal code of Carriage Return char */
#define LF 10            /* Decimal code of Line Feed char */
#define EOF_MARKER 26    /* Decimal code of DOS end-of-file marker */
#define MAX_REC_LEN 1024 /* Maximum size of input buffer */

/* Read functions */
int T_fscanf(FILE *InputFile, char *ReadBuffer);
int T_fgets(FILE *InputFile, char *ReadBuffer);
int T_fgetc(FILE *InputFile, char *ReadBuffer);
int T_fread(FILE *InputFile);

/* Output functions */
void PrintHeader(char *CommandLineArgs[], long FileLength);
void PrintLine(
                char *TextReadFromFile,
                long  CurrentLineNumber,
                long  LengthOfLine,
                long  OffsetOfStartOfCurrentLine,
                long *OffsetOfEndOfPreviousLine,
                int   OffsetError
              );

/* Utility functions */
void HR(int LengthOfHorizontalRule);
void syntax(void);

/*******************************************************************************/
int main(int argc, char *argv[])
/*******************************************************************************/
{
  /* Array of pointers to read functions */
  int (*GetLine[3])(FILE*, char*) = { T_fscanf, T_fgets, T_fgetc };

  int   iReadMode;               /* Index into *GetLine[] array */
  int   iReadReturn;             /* Result of read function */
  int   isFilePosErr;            /* Boolean indicating file offset error */
  long  lFileLen;                /* Length of file */
  long  lLastFilePos;            /* Byte offset of end of previous line */
  long  lLineCount;              /* Line count accumulator */
  long  lLineLen;                /* Length of current line */
  long  lThisFilePos;            /* Byte offset of start of current line */
  char  szReadLine[MAX_REC_LEN]; /* Input buffer */
  FILE *inputFilePtr;            /* Pointer to input file */

  if (argc < 4)                  /* All arguments are required */
  {
    syntax();
    return 1;
  }

  /* Set index into function pointer array based on command line */
  if (strcmp(argv[3], "fscanf") == 0)
    iReadMode = 0;

  else if (strcmp(argv[3], "fgets") == 0)
    iReadMode = 1;
```

```
else if (strcmp(argv[3], "fgetc") == 0)
  iReadMode = 2;

else if (strcmp(argv[3], "fread") == 0)
  iReadMode = 3;

else /* Oops */
{
  syntax();
  return 1;
}

if (strcmp(argv[2], "t") == 0)
  inputFilePtr = fopen(argv[1], "r");  /* Open in TEXT mode */

else if (strcmp(argv[2], "b") == 0)
  inputFilePtr = fopen(argv[1], "rb"); /* Open in BINARY mode */

else /* Oops */
{
  syntax();
  return 1;
}

if (inputFilePtr == NULL )             /* Could not open file */
{
  printf("Error opening %s: %s (%u)\n", argv[1], strerror(errno), errno);
  return 1;
}

fseek(inputFilePtr, 0L, SEEK_END);     /* Position to end of file */
lFileLen = ftell(inputFilePtr);        /* Get file length */
rewind(inputFilePtr);                  /* Back to start of file */

PrintHeader(argv, lFileLen);           /* Print the header info */

/*
*  The implementation of the fread() function in this program is
*  different enough from the other read methods that it's easiest
*  to just call it explicitly and quit.
*/

if (iReadMode == 3)                    /* Use fread() */
{
  iReadReturn = T_fread(inputFilePtr); /* Read the file and print output */
  fclose(inputFilePtr);                /* Close it */
  HR(80);                              /* Print a separator line */
  return (iReadReturn ? 0 : 1);        /* Exit with success or error code */
}

/* At this point, we'll be using fscanf(), fgets(), or fgetc() */

lLineCount   =  0L; /* No lines read yet */
lLastFilePos = -1L; /* So first line doesn't show an offset error */

while (1)
{
  isFilePosErr = 0;                   /* Clear error flag */
  lThisFilePos = ftell(inputFilePtr); /* Offset of start of line */
                                         /* This will not necessarily be */
                                         /* the absolute file offset if  */
                                         /* the file is opened in TEXT   */
                                         /* mode.                        */
```

```
    if (lThisFilePos != lLastFilePos + 1)  /* Set error flag if not next byte */
      isFilePosErr = 1;

    szReadLine[0] = '\0';                  /* Clear buffer for next line */

    /* Read the next line with the appropriate read function */
    iReadReturn = (*GetLine[iReadMode])(inputFilePtr, szReadLine);

    if (iReadReturn < 0)  /* Error reading line */
    {
      /*
      *  Any system error code generated in the read functions is returned
      *  as a negative number so we can use positive numbers to indicate
      *  success. Therefore, we need to 're-negative' it to convert it back
      *  to the original error code.
      *
      *  Error codes are implentation-dependent, but as far as I know, they
      *  are always positive integers.
      */

      printf("Error reading %s: %s (%u)\n",
             argv[1], strerror(-errno), -errno);
      break;
    }

    lLineLen = strlen(szReadLine); /* Get length of line */

    if (lLineLen)                  /* Got some data */
    {
      ++lLineCount;                /* Increment line counter */

      /* Print the line's detail */
      PrintLine(szReadLine, lLineCount, lLineLen,
                lThisFilePos, &lLastFilePos, isFilePosErr);
    }

    if (iReadReturn == 0)          /* End of file reached */
    {
      lThisFilePos = ftell(inputFilePtr); /* EOF offset */
      HR(80); /* Print a separator line */
      printf("EOF at offset %#x (dec %ld)\n", (int)lThisFilePos, lThisFilePos);
      break;
    }

  }/* end while (1) */

  fclose(inputFilePtr); /* Close the file */
  HR(80);               /* Print a separator line */
  return 0;             /* Exit with success code */

} /* end main() */

/*****************************************************************************/
int T_fscanf(FILE *input, char *output) /* Use:       Read text file w/fscanf */
                                        /*                                    */
                                        /* Arguments: FILE *input             */
                                        /*              Pointer to input file */
                                        /*            char *output            */
                                        /*              Read buffer           */
                                        /*                                    */
                                        /* Return:    int                     */
                                        /*               0 = end of file      */
                                        /*              >0 = # of fields read */
```

```
/*****************************************************************************/
{
  /*
  *  The fscanf() function has some limitations that usually make it
  *  inappropriate for reading text files, the main one being that
  *  it will stop reading at the first space character. It is included
  *  here for completeness.
  */

  int iReturn = fscanf(input, "%s", output); /* Read from file */

  if (iReturn == EOF) /* End of file reached */
    return 0;

  return iReturn;

} /* end T_fscanf() */

/*****************************************************************************/
int T_fgets(FILE *input, char *output) /* Use:       Read next line of text   */
                                       /*            file with fgets          */
                                       /*                                     */
                                       /* Arguments: FILE *input              */
                                       /*              Pointer to input file  */
                                       /*            char *output             */
                                       /*              Read buffer            */
                                       /*                                     */
                                       /* Return:    int                      */
                                       /*              <0 = error             */
                                       /*               0 = end of file       */
                                       /*               1 = line read okay    */
/*****************************************************************************/
{
  /*
  *  The fgets() function will read up to 'MAX_REC_LEN' characters
  *  (1K in this program), but will stop at the first newline
  *  (which is a LF in the three compilers tested).
  *
  *  If the line length is greater than 'MAX_REC_LEN', we won't get
  *  the entire line. A real application should take this into account.
  */

  fgets(output, MAX_REC_LEN, input); /* Read the line */

  if (ferror(input))                 /* Error reading */
    return -errno;                   /* Convert code to negative number */

  if (feof(input))                   /* End of file reached */
    return 0;

  return 1;

} /* end T_fgets() */

/*****************************************************************************/
int T_fgetc(FILE *input, char *output) /* Use:       Read next line of text   */
                                       /*            file with fgetc          */
                                       /*                                     */
                                       /* Arguments: FILE *input              */
                                       /*              Pointer to input file  */
                                       /*            char *output             */
                                       /*              Read buffer            */
                                       /*                                     */
                                       /* Return:    int                      */
```

```
                                        /*              <0 = error              */
                                        /*               0 = end of file        */
                                        /*               1 = line read okay     */
/*****************************************************************************/
{
  /*
  *  This function repeatedly calls fgetc(), reading one character at
  *  a time, until it encounters the first character FOLLOWING a CR
  *  or LF that is NOT a CR or LF (or reaches the end of file). It
  *  assumes that everything up to (but NOT including) this character
  *  is part of the current line.
  *
  *  As a result, this function will NOT read BLANK lines correctly.
  *  It will include ALL CRs and LFs as the trailing characters
  *  on the current line. A real application should take this into
  *  account.
  *
  *  This function works okay, but reading a file one byte at a time
  *  is rather inefficient. See the T_fread() function in this program
  *  for a better approach.
  */

  int  iReturn   = 1;  /* Return value (Innocent until proved guilty) */
  int  iThisChar;      /* Current character */
  int  isNewline = 0;  /* Boolean indicating we've read a CR or LF */
  long lIndex    = 0L; /* Index into read buffer */

  while (1) /* Will exit on error, end of line, or end of file */
  {
    iThisChar = fgetc(input);     /* Read the next character */

    if (ferror(input))            /* Error reading */
    {
      iReturn = -errno;           /* Convert to negative number */
      break;
    }

    if (iThisChar == EOF)         /* End of file reached */
    {
      /*
      *  If we've already read characters on this line put the EOF back
      *  into the stream (ungetc()). We'll end on the NEXT call to this
      *  function.
      */

      if (lIndex > 0)
        ungetc(iThisChar, input);

      else           /* Nothing read but EOF; we're done with the file */
        iReturn = 0;

      break;
    }

    if (!isNewline) /* Haven't read a CR or LF yet */
    {
      if (iThisChar == CR || iThisChar == LF) /* This char IS a CR or LF */
        isNewline = 1;                        /* Set flag */
    }

    else            /* We've already read one or more CRs or LFs */
    {
      if (iThisChar != CR && iThisChar != LF) /* This char is NOT a CR or LF */
      {
```

```
        ungetc(iThisChar, input);             /* Put char back in stream */
        break;                                /* Done reading this line */
      }
    }

    output[lIndex++] = iThisChar;             /* Put char in read buffer */

  } /* end while (1) */

  output[lIndex] = '\0';                      /* Terminate the read buffer */
  return iReturn;

} /* end T_fgetc() */

/*************************************************************************/
int T_fread(FILE *input) /* Use:       Read text file using fread()      */
                         /*                                              */
                         /* Arguments: FILE *input                       */
                         /*              Pointer to input file           */
                         /*                                              */
                         /* Return:    int                               */
                         /*              0 = error                       */
                         /*              1 = success                     */
/*************************************************************************/
{
  /*
  *  This function reads the ENTIRE FILE into a character array and
  *  then parses the array to determine the contents of each line.
  *  This is lightning-fast, but may not work for large files. (See the
  *  notes preceding the call to calloc() in this function.)
  *
  *  This routine combines the functionality of the main() and T_fgetc()
  *  functions in this program (although, unlike T_fgetc(), it parses
  *  the lines from memory rather than directly from disk). I wrote it
  *  this way so I could keep everything in one source file and easily
  *  share the output routines.
  *
  *  As in the T_fgetc() function, this function will "collapse" any
  *  blank lines. This may not be appropriate in a real application.
  */

  int   isNewline;              /* Boolean indicating we've read a CR or LF */
  long  lFileLen;               /* Length of file */
  long  lIndex;                 /* Index into cThisLine array */
  long  lLineCount;             /* Current line number */
  long  lLineLen;               /* Current line length */
  long  lStartPos;              /* Offset of start of current line */
  long  lTotalChars;            /* Total characters read */
  char  cThisLine[MAX_REC_LEN]; /* Contents of current line */
  char *cFile;                  /* Dynamically allocated buffer (entire file) */
  char *cThisPtr;               /* Pointer to current position in cFile */

  fseek(input, 0L, SEEK_END);  /* Position to end of file */
  lFileLen = ftell(input);     /* Get file length */
  rewind(input);               /* Back to start of file */

  /*
  *  The next line attempts to reserve enough memory to read the
  *  entire file into memory (plus 1 byte for the null-terminator).
  *
  *  The program will simply quit if the memory isn't available.
  *  This normally won't happen on computers that use virtual
  *  memory (such as Windows PCs), but a real application should
  *  make provisions for reading the file in smaller blocks.
```

```
  *
  *  We could use malloc() to allocate the memory, but calloc()
  *  has the advantage of initializing all of the bits to 0, so
  *  we don't have to worry about adding the null-terminator
  *  (Essentially, every character initially IS a null-terminator).
  *
  *  Note that we don't call the free() function to release the
  *  memory allocated by calloc(). It should not be necessary in
  *  this case because cFile is a local variable and will be
  *  deallocated automatically when this function ends.
  */

  cFile = calloc(lFileLen + 1, sizeof(char));

  if(cFile == NULL )
  {
    printf("\nInsufficient memory to read file.\n");
    return 0;
  }

  fread(cFile, lFileLen, 1, input); /* Read the entire file into cFile */

  lLineCount  = 0L;
  lTotalChars = 0L;

  cThisPtr    = cFile;              /* Point to beginning of array */

  while (*cThisPtr)                 /* Read until reaching null char */
  {
    lIndex    = 0L;                 /* Reset counters and flags */
    isNewline = 0;
    lStartPos = lTotalChars;

    while (*cThisPtr)               /* Read until reaching null char */
    {
      if (!isNewline)               /* Haven't read a CR or LF yet */
      {
        if (*cThisPtr == CR || *cThisPtr == LF) /* This char IS a CR or LF */
          isNewline = 1;                        /* Set flag */
      }

      else if (*cThisPtr != CR && *cThisPtr != LF) /* Already found CR or LF */
        break;                                     /* Done with line */

      cThisLine[lIndex++] = *cThisPtr++; /* Add char to output and increment */
      ++lTotalChars;

    } /* end while (*cThisPtr) */

    cThisLine[lIndex] = '\0';     /* Terminate the string */
    ++lLineCount;                 /* Increment the line counter */
    lLineLen = strlen(cThisLine); /* Get length of line */

    /* Print the detail for this line */
    PrintLine(cThisLine, lLineCount, lLineLen, lStartPos, NULL, 0);

  } /* end while (cThisPtr <= cEndPtr) */

  HR(80); /* Print a separator line */
  printf("Length of file array=%#x (dec %d)\n", strlen(cFile), strlen(cFile));

  return 1;

} /* end T_fread() */
```

```
/*****************************************************************************/
void PrintHeader(char *argv[], long lFileLen) /* Use:       Print header info */
                                              /*                              */
                                              /* Arguments: char *argv[]      */
                                              /*             Command line args*/
                                              /*            long lFileLen     */
                                              /*             Length of file   */
                                              /*                              */
                                              /* Return:    void              */
/*****************************************************************************/
{
  HR(80); /* Print a separator line */

  /*
  *  lFileLen is cast to an (int) for display as a hex number because
  *  the Borland compiler couldn't handle converting a (long) to hex.
  *  Visual C++ and gcc were able to handle (long)s. The hex display
  *  will be screwed up if the file size is larger than the maximum
  *  signed (int).
  */

  printf(
          "File=%s, Size=%#x (dec %ld), Open mode=%s%s%s\n",
          argv[1],
          (int)lFileLen,
          lFileLen,
          (strcmp(argv[2], "t") == 0 ? "Text" : "Binary"),
          (argv[3] == NULL ? "" : ", Read mode="),
          (argv[3] == NULL ? "" : argv[3])
        );

  return;

} /* end PrintHeader() */

/*****************************************************************************/
void PrintLine(char *szReadLine,  long lLineCount,    long lLineLen,
               long lThisFilePos, long *lLastFilePos, int  isFilePosErr)
/*****************************************************************************/
/* Use:       Print detail for current line                                  */
/*                                                                           */
/* Arguments: char *szReadLine   = Read buffer containg text line            */
/*            long  lLineCount   = Current line number                       */
/*            long  lLineLen     = Current line length                       */
/*            long  lThisFilePos = Offset of start of current line           */
/*            long *lLastFilePos = Offset of end of current line             */
/*            int   isFilePosErr = True if start of current line is not      */
/*                                 1 greater than end of last line           */
/*                                                                           */
/* Return:    void                                                           */
/*****************************************************************************/
{
  char *cPtr; /* Pointer to current character */

  HR(80); /* Print a separator line */
  printf("LINE %ld, Length=%#x (dec %ld)\n",
         lLineCount, (int)lLineLen, lLineLen); /* See PrintHeader() for an    */
                                               /* explanation of why the cast */
                                               /* is needed.                  */
  printf(" Offset:");

  cPtr = szReadLine; /* Point to start of string */
```

```
  if (isFilePosErr)                 /* Indicates offset error */
    printf("*%2x", lThisFilePos++);    /* Print '*' plus starting offset */
  else                              /* Offset okay */
    printf("%3x", lThisFilePos++);     /* Just print starting offset */

  for (++cPtr; cPtr < szReadLine + lLineLen; cPtr++) /* Remaining offsets */
    printf("%3x", lThisFilePos++);

  if (lLastFilePos != NULL)           /* Set end position if arg passed */
    *lLastFilePos = lThisFilePos - 1;

  printf("\n Hex:   ");

  /* Print the hex values, including null terminator */
  for (cPtr = szReadLine; cPtr <= szReadLine + lLineLen; cPtr++)
    printf("%3x", *cPtr);

  printf("\n Char:  ");

  /* Print the characters, including null terminator */
  for (cPtr = szReadLine; cPtr <= szReadLine + lLineLen; cPtr++)
  {
    switch (*cPtr)
    {
      case 0:                 /* Null terminator */
        printf(" \\0");
        break;

      case CR:                /* Carriage return */
        printf(" cr");
        break;

      case LF:                /* Line feed */
        printf(" lf");
        break;

      case EOF_MARKER:        /* DOS end-of-file marker */
        printf(" em");
        break;

      default:                /* A 'real' character */
        printf("%3c", *cPtr);
        break;

    } /* end switch (*cPtr) */

  } /* end for (cPtr) */

  printf("\n");
  return;

} /* end PrintLine()

/*************************************************************************/
void HR(int iLen) /* Print a horizontal line of iLen length                */
/*************************************************************************/
{
  int i;

  for (i = 0; i < iLen; i++)
    printf("-");

  printf("\n");
  return;
```

```
} /* end HR() */

/*************************************************************************/
void syntax(void) /* Print correct command line syntax                    */
/*************************************************************************/
{
  printf("\nSyntax: READ FileName OpenMode ReadMode\n\n");
  printf("  OpenMode = \"t\" (text mode),  or\n");
  printf("             \"b\" (binary mode)\n\n");
  printf("  ReadMode = \"fscanf\", or\n");
  printf("             \"fgets\",  or\n");
  printf("             \"fgetc\",  or\n");
  printf("             \"fread\"\n");
  return;

} /* end syntax() */

/* end read.c */
```

< Back to *The Program* | ^ Up to Top | Forward to *Program Output* >